

PhasePoly: An Optimization Framework for Phase Polynomials in Quantum Circuits

Zihan Chen
zihan.chen.cs@rutgers.edu
Rutgers University
Piscataway, NJ, USA

Henry Chen
hc867@scarletmail.rutgers.edu
Rutgers University
Piscataway, NJ, USA

Yuwei Jin
yuwei.jin@jpmchase.com
Global Technology Applied Research,
JPMorganChase
New York, NY, USA

Minghao Guo
minghao.guo@rutgers.edu
Rutgers University
Piscataway, NJ, USA

Enhyeok Jang
enhyeok.jang@yonsei.ac.kr
Yonsei University
Seoul, South Korea

Jiakang Li
jiakang.li@rutgers.edu
Rutgers University
Piscataway, NJ, USA

Caitlin Chan
ccc227@scarletmail.rutgers.edu
Rutgers University
Piscataway, NJ, USA

Won Woo Ro
wro@yonsei.ac.kr
Yonsei University
Seoul, South Korea

Eddy Z. Zhang
eddy.zhengzhang@gmail.com
Rutgers University
Piscataway, NJ, USA

Abstract

Quantum computing has transformative computational power to make classically intractable computing feasible. As the algorithms that achieve practical quantum advantage are beyond manual tuning, quantum circuit optimization has become extremely important and integrated into today’s quantum software stack. This paper focuses on a critical type of quantum circuit optimization – phase-polynomial optimization. Phase polynomials represents a class of building-block circuits that appear frequently in quantum modular exponentials (the most time-consuming component in Shor’s factoring algorithm), in quantum approximation optimization algorithms (QAOA), and in Hamiltonian simulations. Compared to prior work on phase polynomials, we focus more on the impact of phase polynomial synthesis in the context of whole-circuit optimization, from single-block phase polynomials to multiple block phase polynomials, from greedy equivalent sub-circuit replacement strategies to a systematic parity matrix optimization approach, and from hardware-oblivious logical circuit optimization to hardware-friendly logical circuit optimization. We also provide a utility of our phase polynomial optimization framework to generate hardware-friendly building blocks. Our experiments demonstrate improvements of up to 50%—with an average total gate reduction of 34.92%—and reductions in the CNOT gate count of up to 48.57%, averaging 28.53%, for logical circuits. Additionally, for physical circuits, we achieve up to 47.65% CNOT gate reduction with an average reduction of 25.47% across a representative set of important benchmarks.

Keywords

Quantum Computing, Circuit Optimization, Phase Polynomials

1 Introduction

Quantum computing (QC) has the transformative capability to tackle certain classically intractable problems such as large number factoring [48], discrete logarithms [42], database searches [19,

49], and physics/chemistry [8, 15, 37] simulations. Current quantum computers are subject to their inherent noise and limited capacities [17] which makes designing efficient circuits for noisy intermediate-scale quantum (NISQ) devices critical.

Circuit optimization is a technique that transforms an input circuit into a semantically equivalent but more efficient circuit. The primary objective of optimizing quantum circuits is to minimize the number of gates and the circuit depth, which is crucial for reducing error rates and execution time, thereby mitigating decoherence and improving the fidelity of quantum operations. Due to the importance of circuit optimization for NISQ devices, circuit optimization has been incorporated as a part of industrial-strength software frameworks such as in Qiskit [1], Quilc [52], and t|ket> [51].

Our Work: We focus on a special type of circuit optimization – *phase polynomial* optimizations. Phase polynomial is a building block circuit that consists of only CNOT and R_z rotation gates, and an example is shown in Fig. 1 (a). Phase polynomials appear frequently in critical quantum applications [42, 56] including Shor’s factoring algorithm [48], quantum oracle construction, QAOA [14], and the product formula approach to Hamiltonian simulations [5, 35]. A phase polynomial circuit can be transformed as long as the R_z rotation gates are applied under proper parity conditions.

For instance, in Fig. 1 (a), the $R_z(\pi/2)$ is applied when $x \oplus y = 1$, hence it is applied to the first qubit right after the first CNOT gate. Note that a CNOT gate applied to $|a, b\rangle$ returns $|a, a \oplus b\rangle$. This implies that the control qubit remains unchanged and the target qubit is an XOR of the control and target. Additionally, for the output, it should also satisfy the parity conditions. For instance, in Fig. 1 (a), the output on the third line of the qubit should be $x \oplus y \oplus z$, assuming that the input is x, y, z . As long as these two conditions are satisfied, we can transform any phase polynomial circuit. Fig. 1 (b) is such an example, where 3 R_z gates and 1 CNOT are reduced, compared with Fig. 1 (a). Phase polynomials are formally represented in the *sum-over-paths* form [2] which we discuss later.

How Our Work Differs from Prior Work: Prior works on phase polynomial optimization typically look at the synthesis of a

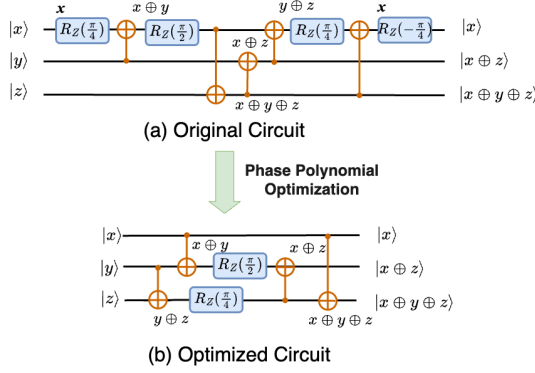


Figure 1: An example phase polynomial circuit. Each CNOT calculates an XOR of the control and target qubit and stores the result in the target qubit with the control remaining the same. A circuit transformation is valid as long as every R_z gate is performed under a proper XOR sum and the output basis follows a correct XOR relationship of the input basis. (a) and (b) are two equivalent circuits, except that the original circuit (a) has 5 CNOT gates and 4 R_z gates, while the optimized circuit (b) has only 2 R_z rotation gates and 4 CNOT gates.

single block of phase polynomials. In this paper, we systematically explore the impact of phase polynomial optimization at the full-circuit level. We investigate the full spectrum of flexibilities in phase polynomial synthesis for circuit optimization, as well as for building hardware-friendly phase polynomials. We provide an enhanced understanding of phase polynomials and lightweight heuristic algorithms without resorting to exhaustive searches. The differences between our work and prior work are as follows:

1. Comparison with phase-polynomial-only optimizations:

Most such studies focus on a single block of phase polynomials. These approaches [2, 55] have investigated different heuristics. For instance, Gray-Synth [2] is inspired by Gray Code to manipulate the parity matrix representation of phase polynomials. Vandaele *et al.* [55] use minimum spanning arborescence to determine the synthesis order of the parity items. However, a general-purpose circuit may consist of multiple blocks of phase polynomials. Existing approaches primarily address optimizations within individual blocks but not at the full-circuit level. Integrating phase polynomial optimization into a holistic optimization framework and optimizing across different phase polynomial blocks remains a challenge.

2. Comparison with the whole-circuit optimization frameworks:

Most existing studies for whole circuit optimization [34, 59, 60] relies on equivalent-circuit rewriting. It first generates a database of equivalent subcircuits, where it enumerates all possible circuits under a qubit-count/depth limit and groups them into semantically equivalent classes. During the optimization stage, it searches for and replaces sub-circuits with their optimal counterparts in the database. This peephole optimization process gradually improves a circuit until it converges. Quartz [60], Queso [59], and Quarl [34] belong to this category.

The closest one to our phase polynomial optimization is the Queso framework, which also generates equivalent classes for phase

polynomial circuits. However, due to the prohibitive overhead of enumerating all possible equivalent sub-circuits corresponding to a phase polynomial, it faces a scalability challenge, as will be shown in later sections of this paper. On the other hand, Nam *et al.* [42] introduce a full-circuit optimizer that considers phase polynomial optimization, but it only considers a restricted set of phase polynomial optimization, including rotation merging and floating. It does not alter the structure of CNOT networks or reduce them, and it does not exploit the full potential of phase polynomials.

3. Comparison with logical-circuit-only optimizations: Most

quantum circuit optimizers [34, 42, 59, 60] do not exploit the flexibility of phase polynomials that could facilitate further hardware mapping, such as reducing the number of SWAPs. While better logical circuits typically translate to better physical circuits in the downstream compilation, we emphasize that it is important to enhance the synergy between logical-level circuit optimization and physical-level circuit translation. The phase polynomial optimization idea itself has the potential to alter the structure of CNOT networks which may help better tackle the sparse hardware connectivity problem [26, 38, 43, 55]. None of the existing whole-circuit optimizers has exploited this property.

Key Takeaway Message: *Our work is the first study that systematically explores phase polynomials for holistic whole-circuit optimization.* **Our contributions** are summarized as follows:

- *Enhanced understanding of phase polynomial synthesis and its impact on quantum circuit optimizations:* (1) We, for the first time, enabled synthesis across multiple phase polynomial blocks. (2) We improved single-block optimization with a joint parity matrix of phase rotation and basis transformation. (3) We propose an intermediate representation (IR) design that facilitates and capitalizes on the gains of multi-block merging optimization.
- *Orthogonality to other approaches:* Our phase polynomial optimizer, as a standalone pass, can be integrated into a circuit optimization framework. For instance, we can apply our phase polynomial optimization pass first to exploit long-range optimization opportunities, and then apply an equivalent-circuit-rewriting optimization pass to catch short-range optimization opportunities. It is complementary to existing quantum circuit optimization passes.
- *Extensibility:* Our phase polynomial optimization algorithm can easily be extended to take hardware connectivity constraints into account. We can produce hardware-friendly phase polynomial blocks that require linear connectivity as opposed to all-to-all, and significantly reduce the burden on hardware qubit mappers that happen later in the physical circuit compilation.
- *Results:* Our phase polynomial optimization framework achieves up to 50% total gate reduction (averaging 34.92%) and up to 48.57% CNOT reduction (averaging 28.53%) in logical circuits. When integrated with search-based rewrite frameworks, it outperforms existing methods, especially on large circuits. For physical circuits, our method outperforms 82.35% of benchmarks, achieving up to 47.65% CNOT reduction (averaging 25.47%).

2 Background and Motivation

A quantum circuit is a sequence of quantum gates operating on an n -qubit system. A universal quantum gate set that can implement any quantum algorithm comprises CNOT gates and single-qubit rotation gates. The gates operate on a superposition of the computational basis states, which are denoted by $|x\rangle$, where $x \in \mathbb{F}_2^n$ is a binary vector with exactly one entry equal to 1. For other basics in quantum computing, we refer the readers to the introduction by Nielson and Chuang [44].

A CNOT gate acts on two qubits a and b such that $\text{CNOT}|a, b\rangle \rightarrow |a, a \oplus b\rangle$, where a is called the control qubit and b is called the target qubit. The control qubit remains unchanged, and the target qubit is an XOR ($\oplus$) of the control and target qubits.

Taking the R_z (Z-rotation) gate from the single-qubit rotation gates and the CNOT gate, one can form a phase-polynomial circuit [9, 40]. Two examples are shown in Fig. 1. This type of circuit can be expressed in a *sum-over-paths* form [2, 3, 28] such that

$$U|x_1, \dots, x_n\rangle = e^{ip(x_1, \dots, x_n)}|g(x_1, \dots, x_n)\rangle, \quad (1)$$

where $p(x_1, \dots, x_n)$ is a polynomial of XOR operations with phase rotation angles as the coefficient.

$$p(x_1, \dots, x_n) = \sum_{y \in \{0,1\}^n} \theta_i (x_1 y_1 \oplus \dots \oplus x_n y_n).$$

For example, the polynomial for the example in Fig. 1 is $p(x, y, z) = \frac{\pi}{2}(x \oplus y) + \frac{\pi}{4}(y \oplus z) + (\frac{\pi}{4} + \frac{-\pi}{4})x = \frac{\pi}{2}(x \oplus y) + \frac{\pi}{4}(y \oplus z)$.

The other component of phase polynomials is the basis transformation $g(x)$, where in the example in Fig. 1, the input basis $|x, y, z\rangle$ is transformed to the output basis $|x, x \oplus z, x \oplus y \oplus z\rangle$. It corresponds to the $g(x, y, z)$ function in Equation (1). It is an affine reversible mapping, achieved by CNOT networks.

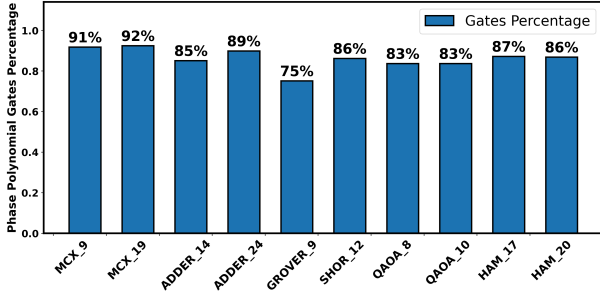


Figure 2: Percentage of phase polynomial gates in quantum circuits, and improvement of our PhasePoly framework. The blue bars represent the ratio of ($\{R_z^\theta, CX\}$) gates to the total gate count, and the circuit names include the number of qubits.

2.1 Why Study Phase Polynomials?

There are a substantial number of phase polynomials in quantum circuits. We demonstrate this by using a representative set of quantum circuits, including multi-controlled NOT gates (MCX), Grover’s Search [19], Shor’s Factoring [49] algorithm, the quantum approximate optimization algorithm (QAOA), and Hamiltonian dynamics

simulations (HAM) in Fig. 2. These benchmarks are commonly used for circuit optimization studies [42, 59, 60]. Our analysis shows that over 75% of gates in these important benchmarks are CNOT and R_z gates, and for some benchmarks, the percentage exceeds 90%. This underscores the importance of building a systematic optimizer for phase polynomials.

2.2 Beyond Local Equivalent-Circuit Rewriting

Phase polynomials constitute a large portion of modern quantum circuits. Now, the question is: how do we perform optimizations? The prevailing approach is the equivalent subcircuit rewriting approach. The idea is to find a candidate subcircuit and replace it with a semantically equivalent but better subcircuit in terms of gate count and depth. This process continues replacing subcircuits until no further optimizations are possible. Typically, this approach involves the generation of an equivalent-subcircuit database [34, 59, 60], which usually requires brute force computation. Hence, the size of these types of subcircuits is small. The phase polynomial version of this is Queso [59] which also generates equivalent circuits based on phase polynomial representations as in Equation 1 in addition to generating non-phase-polynomial equivalent subcircuits.

Our approach proposed in this paper is an optimizer based on parity matrix manipulation for single-block and cross-block phase polynomial optimization (described in Section 3) which can handle longer-range circuit optimizations for phase polynomial forms. We compare our approach with the equivalent-subcircuit-rewriting approach, especially at larger scales, using the multi-controlled NOT (MCX) circuit as a benchmark. The MCX gate, along with its generalized form as a multi-controlled unitary operation, is central to nearly all quantum algorithms and simulations [66]. Such multi-controlled operations appear in applications ranging from particle number-conserving Hamiltonians for quantum neuron implementations [4, 11] and quantum circuits for isometries [21, 36] to distributed quantum computing [54], Grover’s search algorithm [19], and the arithmetic component of Shor’s algorithm [49].

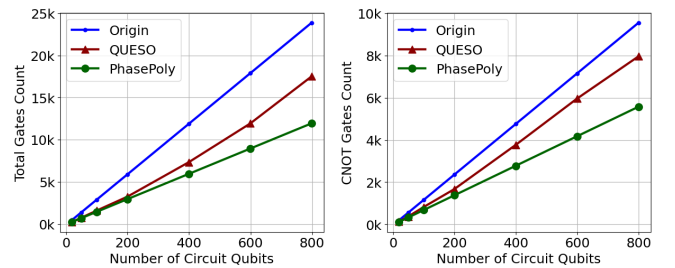


Figure 3: Gate count comparison across different frameworks for optimizing multi-controlled operations.

We show the results in Fig. 3 for MCX benchmarks with qubit counts from 19 to 799, and the overall original gate counts from 450 to 24,000. Our comparisons show that while Queso can also reduce gate counts with respect to the original version, our optimizer achieves greater reductions, especially when the circuit size is large. Moreover, we selected a version of the circuit that employs R_z gate preprocessing to ensure Queso achieves optimal performance;

however, even under these conditions, Queso never outperforms our approach on MCX circuits. We noticed that the CNOT gate count decreases linearly for both our approach and Queso. Queso can also catch these cross-block optimizations using its equivalent circuit database, but at relatively smaller ranges. Our optimization can catch more opportunities since we allow the merging of blocks and can optimize in the range of two (or more) blocks with over 20 gates. However, it is difficult for Queso to manage a subcircuit size of 20 gates or more due to the size of its subcircuit classes [59].

We also want to point out that Queso has other non-local phase polynomial optimizations enabled, not just focusing on phase polynomials. However, our approach still outperforms Queso when the number of qubits exceeds 100, and the performance gap between our approach and the Queso approach widens as the number of qubits increases.

2.3 A Useful Utility: Hardware-Friendly Logical Circuit Optimization

In current hardware, sparse connectivity poses significant challenges for logical-to-physical circuit translation, which tends to require SWAP gate insertion. A two-qubit gate can run on two physically connected qubits. Hence, logical qubits may need to be moved around using SWAP gates before a two-qubit gate can be executed. Logical circuit optimization and logical-to-physical circuit translation are typically two isolated processes in the current quantum software stack. Although it provides a separation of responsibilities in compilation, it may lack consideration of synergistic optimization opportunities. We still use the example of the MCX benchmark. The pattern of the MCX circuit is shown in Fig. 4 (c), where it uses sequences of Toffoli gates.

The original Toffoli gate using the Clifford and T implementation [44], as shown in Fig. 3 (a), requires an all-to-all connectivity for its three qubits. However, it is rare for any modern hardware to have triangles in its architecture, whether it is a rotated lattice in the fault-tolerant case or a heavy-hex architecture in large-scale IBM quantum processors. This decomposition of the Toffoli gate, when embedded in the whole MCX circuit, may pose a significant burden for further SWAP insertions.

Instead, using our hardware-friendly logical circuit optimizer, we can turn the all-to-all-connectivity requirement of Toffoli gate into a linear-connectivity requirement, as shown in Fig. 4 (b), where although the total number of CNOT gates is increased from 6 to 8, the requirement for clique connectivity is removed. We employ our Toffoli decomposition into the original MCX circuit, perform single-qubit phase polynomial optimizations, and compare that with the original Toffoli decomposition version.

It can be seen from Fig. 5 that the total CNOT counts after SWAP insertion (one SWAP is counted as 3 CNOT gates as in prior work [31]) from our version of embedding linear connectivity Toffoli is much better than the original version. Moreover, we also compare with Queso + Sabre, which is the version that first optimizes the logical circuit and then runs a general-purpose qubit mapper of the optimized logical circuit. Our approach PhasePoly + Sabre is much better than that as well. The reason is that Toffoli is a frequently appearing building block in the MCX circuit, and hence making

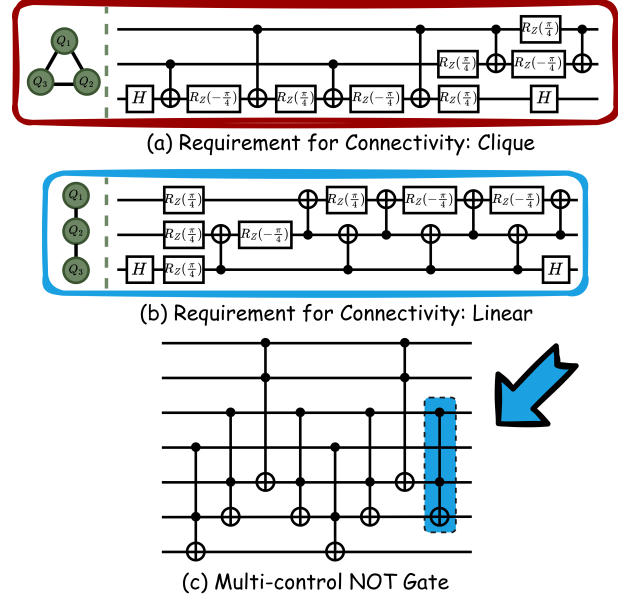


Figure 4: Illustration of hardware-friendly template generation. The figure shows how a Toffoli gate can be converted into a form that only requires linear connectivity, while the original Toffoli decomposition [44] clique connectivity. The linear Toffoli template alleviates the burden of the hardware qubit mapper in the downstream compilation. All are used in combination with Sabre.

them hardware-friendly (linear-connectivity-requirement rather than clique-connectivity-requirement) is beneficial.

It is worth noting that a previous study, Trios [13], has exploited the recurring pattern of the Toffoli gate in quantum circuits and manually came up with a linear-connectivity Toffoli gate decomposition and further proposed a whole-circuit routing approach for the Toffoli and other gates. The difference between our work and Trios is that: (1) We automatically generate linear-connectivity circuit blocks. As will be shown later in the evaluation, we can synthesize not only linear Toffoli, but also other building blocks such as linear 4-qubit SUM/CARRY [57] in VBE adder, which is a

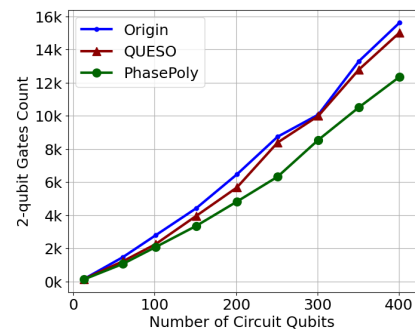


Figure 5: CNOT gate count comparison on limited-connectivity hardware for the MCX circuits.

frequent component in modular exponential for Shor’s algorithm. (2) We do not provide a whole-circuit routing approach directly; instead, we collaborate with the best qubit mappers by embedding the linear-connectivity version of these building blocks into the circuit, as if they are hardware-friendly templates, and hence enable collaboration between upstream and downstream compilations without having to give up the benefit of the division of abstraction layers in the software stack.

3 Phase Polynomials Optimization

We will discuss three key techniques that distinguish our approach from prior work: (1) co-optimization of parity terms for rotation gates and basis transformation, (2) an optimization framework using a memory-bounded A* priority queue, and (3) a grouped multi-block synthesis engine with a corresponding IR design that allows merging of different phase polynomial blocks within a circuit.

3.1 Co-Optimization of the Phase Parity Gates and Basis Transformation

Recall that there are two components in a phase polynomial representation: 1) the component that executes a phase gate $e^{p(x)}$ conditional on the input basis $|x\rangle$, and 2) the component that transforms the input basis vector: $|x\rangle \mapsto |g(x)\rangle$. Using the same example in Fig. 1, we have

$$|x, y, z\rangle \rightarrow e^{ip(x,y,z)} |x, x \oplus z, x \oplus y \oplus z\rangle, \quad (2)$$

where $p(x)$ is defined as:

$$p(x, y, z) = \frac{\pi}{2}(x \oplus y) + \frac{\pi}{4}(y \oplus z) \quad (3)$$

Prior studies [3, 55] optimize for the $p(x)$ component and $g(x)$ component separately. However, the optimization of one of them may affect another. To achieve the $p(x)$ component on the exponent of e , we need to apply CNOT gates, and that will have an impact on the basis state transformation. For example, in the circuit shown in Fig. 6(a), synthesizing only the phase parity component uses two CNOT and two R_z gates. The resulting output state, $|x, x \oplus y, y \oplus z\rangle$, does not match the desired state from Equation (2), $|x, x \oplus z, x \oplus y \oplus z\rangle$. An additional transformation is required to convert $|x, x \oplus y, y \oplus z\rangle \mapsto |x, x \oplus z, x \oplus y \oplus z\rangle$, which is achieved using a linear reversible circuit of only CNOT gates [10, 39, 45].

Previous works [3, 55] often treat phase parity synthesis and linear reversible circuit synthesis as isolated steps, which may lead to suboptimal overall results. Different phase parity circuits produce different intermediate states, impacting the final linear reversible circuit design. For instance, in Fig. 6, both have the optimal gate count for phase parity (phase gate component), using two CNOT and two R_z gates. However, Fig. 6(a) is superior to Fig. 6(b) with respect to output basis state transformation, requiring only two CNOT gates versus three.

When a phase polynomial circuit optimization is split into two isolated parts – phase parity network synthesis and linear reversible circuit synthesis – **the optimization may significantly deviate from the overall optimum.**

Our approach is to co-optimize both the phase parity network and the linear reversible circuit synthesis. We use a data structure to encode the impact of CNOT gates on both the

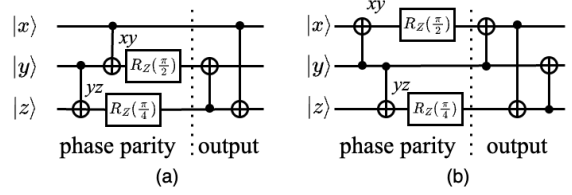


Figure 6: Two functionally equivalent circuits where for the phase parity component, they are equally optimal, but for the output state component, one is better than the other. For illustration purposes, we let the concatenation xy denote $x \oplus y$ in the rest of the paper.

optimization components. It consists of the phase parity matrix and the output parity matrix, such that for each CNOT applied to the phase parity matrix, it also applies to corresponding entries in the output parity matrix. We show such a data structure for the example circuits in Fig. 6 below:

$$\left[\begin{array}{cc|ccc} 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right] \xrightarrow{\text{CNOT}(2,1)} \left[\begin{array}{cc|ccc} 1 & 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 1 \end{array} \right] \quad (4)$$

Each column represents a parity item, and each row represents a qubit. For example, the first column (110) represents $x \oplus y$ and the second column (011) represents $y \oplus z$. The left part of the matrix represents the phase parities. The right part of the matrix represents the output parities.

With a CNOT gate where row_i is the control qubit, and row_j is the target qubit, the transformation is:

$$(\text{row}_i, \text{row}_j) \xrightarrow{(\text{CNOT}_{i,j})} (\text{row}_i \oplus \text{row}_j, \text{row}_j) \quad (5)$$

The operation "CNOT q_i, q_j " produces $q_i \oplus q_j$ stored in the j -th qubit, corresponding to setting the control qubit position in the parity matrix to 0 ($1 \oplus 1$) in the column vector, and keeping the target qubit position unchanged (1) in the parity matrix. In the parity matrix, the location that stores the row sum is the control qubit, where it stores $\text{row}_i \oplus \text{row}_i$, and essentially sets the control to 0, if both row_i and row_j are 1. After applying the CNOT in Equation (4), the intermediate output is $|x \oplus y, y, z\rangle$ as shown in Fig. 6 (b), and the first column vector changes from $[1, 1, 0]^T$ to $[1, 0, 0]^T$. The first column of the parity matrix now has only one "1" at qubit 1, and it is ready to apply a R_z rotation gate. This corresponds to the first CNOT between x and y , and the $R_z(\frac{\pi}{2})$ rotation gate on the first qubit in Fig. 6(b). After this, the column with only one "1" is removed. This process repeats until all columns are removed from the parity matrix. At one time, it removes one or more columns, if this or these columns have only one "1" in it. This approach is not our contribution and is originally from [55].

Our contribution is that we added the output matrix component, shown as the matrix to the right of the vertical line in Equation (4). In our formulation, when a CNOT modifies the phase parity matrix on the left, it will also modify the output matrix on the right. In our initial output matrix – the right part of Equation (4), each column represents a qubit’s output state. For instance, the first qubit x ’s output should be x . Hence, the first column of the output

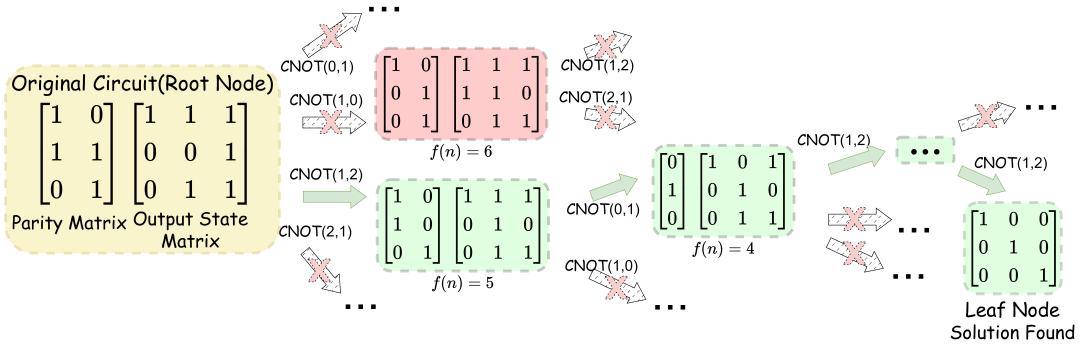


Figure 7: Search tree structure for Phase Polynomial Co-Optimization, where the Root Node represents the input circuit state, the Leaf Node represents the output circuit state, and edges between nodes represent CNOT operations. The cost function below each node indicates the evaluation of the current state in our heuristic-based approach.

matrix is (100). When performing a CNOT (2,1) essentially CNOT (y,x), the parity term in the output matrix (right side of Equation 4) changes from 100 to 110, as the first qubit now reflects $x \oplus y$. As the second qubit is still y, (110) means taking $(x \oplus y) \oplus y = x$, still maintaining the correct output state of the first qubit. After adding this output matrix, our goal is to keep performing row sums until the output matrix becomes an identity matrix, different from the goal of manipulating the phase matrix on the left.

From these observations, we can establish the following rules for CNOT and R_z gate operations in the parity matrix:

- (1) **CNOT Operation:** When a CNOT operation is applied with row_i as the control qubit and row_j as the target qubit, row_j remains unchanged, while row_i is updated to $row_i \oplus row_j$.
- (2) **R_z Operation:** If a column in the parity matrix has only one '1', an R_z gate can be applied to the corresponding qubit state, completing the parity term. After the R_z operation, the column is removed to simplify the phase polynomials.

The optimization problem can be transformed into a **parity CNOT network search problem** [2], with the goal of finding the optimal network that minimizes the total gate count. For the phase parity matrix, the objective is to eliminate columns sequentially, while for the output parity matrix, the goal is to transform it into an identity matrix, reducing each column to a single 1 stored in the corresponding qubit. That basically means each qubit has the correct output stored in itself.

With this search model, we evaluate the impact of each CNOT operation on both the phase and output parity components, aiming to find a sequence of CNOT gates that transform the matrix into the desired form. This new data structure forms the foundation of our memory-bounded A* priority queue optimization framework.

3.2 The Overall Optimization Framework

Parity network synthesis is proven to be NP-hard [2], making it challenging to find the optimal solution in polynomial time. We propose a heuristic-based approach to optimize the sequence of CNOT and R_z operations to minimize the total gate count.

We model the search space as a tree, where each node represents a quantum circuit state, and each edge corresponds to a CNOT

operation, as shown in Fig. 7. A path from the root to a leaf node provides a complete sequence of CNOT operations. The leaf node must have an empty phase parity matrix and an identity matrix for the output state. The objective is to minimize the CNOT count, corresponding to the path with the least cost.

3.2.1 Active Row Pair CNOT Selection. As described in Section 3.1, a CNOT operation corresponds to an XOR operation between two rows in the parity matrix. We refer to it as a **Row Pair**. This operation is directional: a row pair (row_i, row_j) represents a CNOT where row_i is the control qubit and row_j is the target qubit. The result is that the control qubit row is updated as $row_i \oplus row_j$, while the target qubit row remains unchanged.

A row pair is considered valid if there exists at least one column for which performing the row pair XOR operation can reduce the Hamming weight (the number of 1s) in that column. We refer to such pairs as **Active Row Pairs**. However, focusing exclusively on row pairs can lead to livelock, where we repeatedly use the same row pair without finding an optimal solution or getting stuck in a loop. To prevent this, we introduce the **Active Columns Set**, which limits the search space for each row pair. **Row pairs** are only considered if they can reduce the Hamming weight for at least a column within the **Active Column Set**. The active column set itself is updated when necessary after a CNOT.

At the beginning of the phase polynomial optimization, or after completing a parity term (removing that column from the parity matrix), **Active Column Set** is initialized with all remaining columns in the parity matrix.

3.2.2 The Priority Queue Implementation. To evaluate the state of each move after applying a CNOT from the active row pair set, we derive the following insights:

- **Phase parity cost estimation $h_1(n)$:** Sum of the Hamming weight (number of 1s) in the updated phase parity matrix correlates with the number of CNOT gates required for phase gates.
- **Output parity cost estimation $h_2(n)$:** The number of CNOT operations needed for the Gaussian Elimination process for the output state transformation.

- **Actual cost from the root $g(n)$:** The cumulative CNOT gate count from the root node to the current state represents the actual cost incurred so far.

Hence, we have the cost function $f(n)$:

$$f(n) = g(n) + h_1(n) + h_2(n) \quad (6)$$

In the A^* search, we use a priority queue to store circuit states, where the priority of each state is determined by the total cost function $f(n)$. The algorithm explores potential circuit states, and when it encounters an empty phase parity matrix, it finishes the synthesis using the Gaussian Elimination (the minimum degree-based Gaussian elimination). Otherwise, it applies CNOT operations from the active row pair sets and inserts new successor states into the priority queue. An example is illustrated in Fig. 7.

During this process, there may be times when multiple states have the same total cost. To resolve this, we use the lexicographical ordering of a tuple defined as

$$[f(n), h_1(n), h_2(n), -g(n)]$$

as the **tie breaker** to determine which state to prioritize. As the search space grows exponentially larger, the complexity becomes impractical for quantum circuit compilation. To mitigate this, we implement a **Space Bounded A^* Search**. Similar to the Memory-Bounded A^* Search [25, 63], we cap the size for the priority queue. When the queue exceeds this limit, we drop the lower priority nodes from the queue until they meet the queue size requirement, thereby pruning less promising paths and preventing further exploration along suboptimal directions.

Additionally, we implement a **Multiple Solution Search** mechanism controlled by a hyperparameter k . When the algorithm reaches a goal state, the state is added to a solution set of size k . The search continues until either the priority queue is empty or k solutions have been found, at which point the best solution from the set will be returned.

3.3 Multi-block Optimization and IR Design

Since general-purpose circuits include single-qubit gates other than R_z , we partition the circuit into blocks of phase polynomial sub-circuits separated by non- R_z gates. An example of three such blocks is shown in Fig. 8(a).

Larger phase polynomial blocks offer more optimization opportunities, especially when common parity sub-terms appear across different blocks. For instance, in Fig. 8(a), the parity term $q_0 \oplus q_2$ appears in both the first and third blocks, while q_0 and q_2 remain unchanged across all blocks, with only q_3 varying. If we synthesize each block independently, the circuit would require 5 R_z gates and 10 CNOT gates. However, by reusing the $q_0 \oplus q_2$ term, as shown in Fig. 8(b), we reduce the CNOT count to 8. This is achieved through our renaming technique and merging of the blocks.

We introduce a new intermediate representation (IR) that renames qubits similarly to static single assignment (SSA) in classical compilation. Each qubit is renamed when encountering non- R_z (or non- X) gates. For example, q_3 becomes q'_3 after the first Hadamard gate and q''_3 after the second. CNOT gates update the target qubit based on previous qubit states. Renaming enables consecutive phase polynomial blocks to be merged, as shown in Fig. 8.

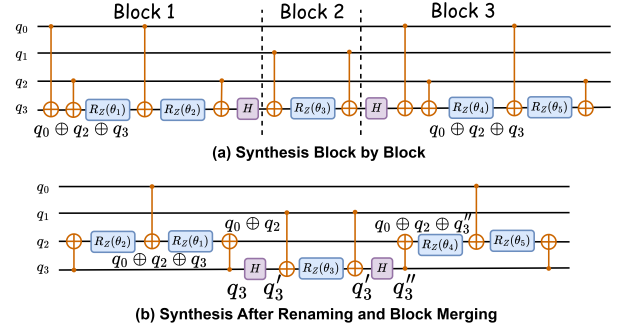


Figure 8: Multi-block phase polynomials: (a) Without renaming, the circuit is divided into three blocks, limiting cross-block optimization. The term $q_0 \oplus q_2$, even if it did not change, cannot be reused due to block boundaries. (b) After renaming, the blocks merge into a single phase polynomial block, reducing CNOT gates from 10 to 8 through the reuse of $q_0 \oplus q_2$.

When multiple blocks are merged, a new parity matrix is generated. Though multiple qubit lines correspond to the old and new states of one original qubit line due to renaming, only one row of the parity matrix corresponds to a qubit line at any time. Once a row is reduced to zero, another row for that qubit line becomes active. Dependence rules ensure that gates involving earlier states of the same qubit state are synthesized before those involving later states, maintaining synthesis correctness. We define a group size k to control how many blocks are merged.

3.4 Phase Polynomial for Physical Circuits

We integrate hardware mapping into our phase polynomial optimization framework by adapting circuit state nodes and generating successor nodes for the priority queue (Section 3). By incorporating qubit mapping into the circuit state, we can search for optimal CNOT and/or SWAP gates to address hardware constraints. As mentioned before, this is a utility of our overall priority-queue based approach.

To demonstrate an example where hardware-aware synthesis is useful, see an example in Fig. 9. In this example, we can synthesize a circuit with 4 CNOT gates and conform to the hardware requirements, if the synthesis is proper. However, the original circuit would require at least one SWAP, if not resynthesizing the phase polynomials it has.

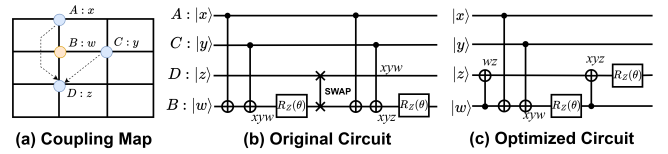


Figure 9: SWAP-free synthesis: (a) Coupling graph; (b) Original circuit needs a SWAP; (c) Synthesis requires no SWAP.

The difference between our logical and hardware-aware phase polynomial optimization is that successor state generation now accounts for qubit mapping. Instead of focusing only on reducing

hamming weight in the active column set, it also considers gates that minimize the distance between interacting qubits. We optimize this using both average and minimum qubit distances.

For **successor state generation**, two key questions arise: (1) determining viable CNOT or SWAP gates that generate candidate circuit state nodes, and (2) adapting the cost function to account for hardware connectivity. This cost function ranks circuit state nodes in the priority queue.

We define the **Active Qubit Set** as qubits with rows of 1's in an active column. For viable gate candidates, we consider only the following three cases:

- **Case (A):** All executable CNOTs from the **Active Row Pair Set** that satisfy the two-qubit connectivity requirement.
- **Case (B):** CNOTs that reduce the distance metric for qubits in the **Active Qubit Set**, even if they don't reduce the Hamming weight of any active column. These gates can entangle qubits in the same column, making them come into a connected component.
- **Case (C):** A SWAP gate is valid if Case (A) does not hold and it reduces the **minimum distance** for at least one active column. The minimum distance refers to the smallest pairwise distance between active qubits in a column.

The definition of the **Active Column Set** now includes columns where either the 1's or the distance metric have been reduced but not fully, or where the minimum distance has improved before any executable active row pair is available.

The distance metric calculates the shortest path between nodes using only qubits within a local coupling graph for each phase polynomial block. In this graph, edges are weighted: visiting an active qubit (with a 1 in the active column) has a weight of 1, while other connections have a SWAP cost of 3 (since a SWAP requires 3 CNOT gates). This reflects the fact that if an active qubit is connected, a CNOT can perform the row sum directly; otherwise, a SWAP is needed. The distance metric is defined as follows:

$$Dist(a, b) = \sum_{e \in (a \rightarrow b)} weight(e_{start}, e_{end}), \quad (7)$$

where $weight(e_{start}, e_{end}) = 3$, if e_{end} is an inactive qubit for the column, otherwise $weight(e_{start}, e_{end}) = 1$. e is an edge on the shortest path from the starting point a to the ending point b .

For the **cost function**, we update it to take into consideration the hardware mapping cost. The $g(n)$ function is still the CNOT gate count cost up until the point. Now we have the new $h'_1(n)$ accounting for the phase parity network and the $h'_2(n)$ accounting for the output parity network. For the phase parity network, we have

$$h'_1(n) = \sum_{p \in C} \sum_{n, j \in qA(p)} Dist(n, j) / (|qA(p)| - 1). \quad (8)$$

C is the set of columns. $qA(p)$ is the active qubit set for p . If qubits have all-to-all connectivity, the sum above is the number of 1's in the current phase parity matrix. This part accounts for the sum of 1's in $h_1(n)$ for logical optimization while also considering the potential SWAP and synthesis cost if qubits are not directly connected. Fig. 10 shows an example for $h'_1(n)$ cost calculation. For $h'_2(n)$, we calculate the cost of all pre-determined CNOT gates for Gaussian elimination (reducing the output matrix to identity) by finding the distance

between the involved qubits. Thus, the cost function is $f(n) = g(n) + h'_1(n) + h'_2(n)$.

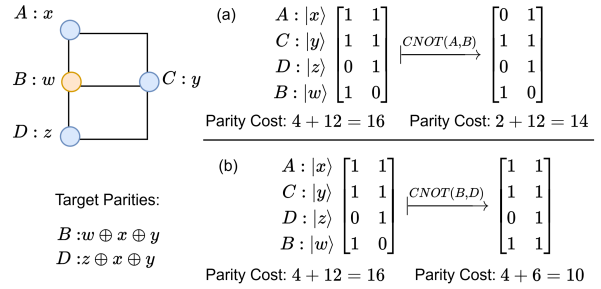


Figure 10: An example to calculate the phase parity cost $h'_1(n)$. In (a), it calculates CNOT(A,B), while it reduces parity cost $h'_1(n)$, it is not as good as (b) CNOT(B,D), which has a smaller cost function. CNOT(B,D) is the one chosen in the motivation example in Fig. 9.

Finally, we adopt an initial mapping optimization similar to Sabre [31]. We run the synthesis and mapping process once, reverse the circuit using the final mapping as the new initial mapping, and perform another pass. This repeats multiple times to search for an effective initial mapping.

4 Evaluation

4.1 Experimental Setup

Benchmarks: In our experimental design, we selected a set of benchmark circuits that have been widely used in previous circuit optimization research, following prior work [2, 3, 20, 27, 34, 42, 47, 59, 60]. In addition, we incorporate new benchmarks of near-term quantum computing applications and future quantum supremacy scenarios. Specifically, our benchmarks encompass datasets for arithmetic circuits, multi-control Toffoli gate implementations, the Hamming coding function, simulations of Hamiltonian dynamics, and implementations of QAOA (Quantum Approximate Optimization Algorithm), Grover's algorithm, and Shor's algorithm.

Metrics: We evaluate our optimization technique using two main metrics: the total gate count and the number of two-qubit CNOT gates (with SWAP gates decomposed into three CNOTs for physical circuits). The reduction rate is calculated as the percentage decrease in gate count relative to the original circuit, while the geometric mean reduction assesses the overall optimization performance.

Baselines: For phase polynomial-only optimization, we compare our framework, PhasePoly, with two existing methods: the Rotation Merging technique proposed by Nam *et al.* [42] and the minimum spanning tree (MST)-based heuristic phase parity network synthesis by Vandaele *et al.* [55]. Both methods are designed for single-block phase polynomial optimization, with the MST-based approach notably outperforming the earlier Gray-Synth framework [2] for randomly generated phase parity network synthesis tasks.

For logical full circuit optimization, we compare our method with GRAY-SYNTH enhanced with additional T gate reduction and flexible CCZ decomposition preprocessing techniques, allowing us to evaluate the relative advantages of our approach against

Circuit	Org.		Rotation Merging		MST-based		PhasePoly.	
	# Gates	# CXs	# Gates	# CXs	# Gates	# CXs	# Gates	# CXs
tof_3	45	18	39	18	39	18	35 (22.22%)	14 (22.22%)
barenco_tof_3	58	24	46	24	46	24	38 (34.48%)	16 (33.33%)
mod5_4	63	28	51	28	49	26	45 (28.57%)	24 (14.29%)
tof_4	75	30	63	30	63	30	55 (26.67%)	22 (26.67%)
tof_5	105	42	87	42	87	42	75 (28.57%)	30 (28.57%)
barenco_tof_4	114	48	86	48	82	44	68 (40.35%)	30 (37.50%)
mod_mult_55	119	48	109	48	98	48	90 (24.37%)	40 (16.67%)
vbe_adder_3	150	70	109	70	85	46	75 (50.00%)	36 (48.57%)
barenco_tof_5	170	72	126	72	118	64	98 (42.35%)	44 (38.89%)
rc_adder_6	200	93	190	93	164	83	152 (24.00%)	71 (23.66%)
tof_10	255	102	207	102	205	100	175 (31.37%)	70 (31.37%)
hwb6	259	116	236	116	227	114	200 (22.78%)	96 (17.24%)
mod_red_21	278	105	246	105	206	103	180 (35.25%)	77 (26.67%)
qaoa_n8_p4	440	96	248	96	248	96	240 (45.45%)	88 (8.33%)
ham15-low	443	236	395	236	384	225	336 (24.15%)	198 (16.10%)
qcla_com_7	443	186	339	186	305	167	256 (42.21%)	120 (35.48%)
barenco_tof_10	450	192	326	192	298	164	248 (44.89%)	114 (40.63%)
qcla_adder_10	521	233	448	233	436	221	391 (24.95%)	176 (24.46%)
grover_5	831	288	711	288	639	280	455 (45.25%)	202 (29.86%)
qcla_mod_7	884	382	730	382	708	369	628 (28.96%)	289 (24.35%)
adder_8	900	409	716	409	658	363	566 (37.11%)	271 (33.74%)
ham15-med	1272	534	1071	534	1024	487	658 (48.27%)	327 (38.76%)
mod_adder_1024	4285	1720	3549	1720	3324	1495	2565 (40.14%)	1217 (29.24%)
ham15-high	5308	2149	4440	2149	4188	1897	2792 (47.40%)	1363 (36.58%)
shor_15_7	36598	14858	33187	14858	31399	13070	31143 (14.91%)	12814 (13.76%)
Geo. Mean Reduction	-	-	18.30%	0.00%	23.30%	7.33%	34.92%	28.54%

Table 1: Comparison of phase polynomial optimization techniques on logical circuits. Using the original circuit as a baseline, alongside Rotation Merging, MST-based heuristic phase parity network synthesis.

phase polynomial optimization methods augmented with other synthesis techniques. In addition, we integrate our orthogonal optimization approach with the equivalent-subcircuit rewriting method QUESO [59], an optimizer that leverages equivalent-circuit rewriting and state-of-the-art search-based techniques, for both phase-polynomial and non-phase-polynomial cases.

For our hardware-friendly circuit optimization evaluation, we compare our method along several perspectives: **1. Only Mapping.** The original circuit is directly processed by the general circuit mapper Sabre [31, 67] without any logical optimization; **2. Only Topology-Aware Synthesis.** Steiner Tree-based synthesis [43, 55] is applied to the original circuit, which only relies on phase polynomial synthesis and is SWAP free; **3. General Optimization and Mapping.** QUESO is used for logical optimization prior to applying Sabre for qubit mapping; **4. Phase Polynomial Integrated Optimization and Mapping.** Combined our orthogonal framework PhasePoly with Queso to showcase the physical circuit performance difference through targeted phase polynomial optimization; and **5. Hardware-Friendly Circuit Generation:** For circuits exhibiting recurring patterns, hardware-friendly templates are generated for specific building blocks, and the resulting physical circuits are compared with those produced by other methods.

Setup: We set the memory-bounded priority queue and solution set sizes based on the synthesis time budget for each benchmark. On an Intel Xeon 2.20GHz CPU, with a queue and solution size of 10,000, 84% of benchmarks are solved within 120 seconds, and all within 400 seconds. We also compare the performance of the

PhasePoly technique with Queso under various time budgets in Section 4.5.

We compare PhasePoly with the default settings of QUESO and Sabre, allocating a 3,600-second time budget per circuit for QUESO. We adopt the recommended rule set of the phase polynomial gate set for QUESO, and Sabre uses a grid graph with square connectivity as its hardware coupling graph. We verify all optimized circuits—except for the oversized mod_adder_1024—using Qiskit [1] and MQT QCEC [7] for unitary comparison and formal verification, including our generated physical circuit templates.

4.2 Logical Circuit Optimization Improvement using Phase Polynomial Optimization

4.2.1 Evaluation of Phase Polynomial Optimization as a Standalone Component. We compare the performance of the PhasePoly technique with two alternative phase polynomial synthesis methods: Rotation Merging and MST-based parity network synthesis, as shown in Table 1. Rotation Merging focuses exclusively on reducing rotation gates without resynthesizing the circuit or optimizing the CNOT network, resulting in a limited impact on the total gate count. MST-based parity network synthesis includes the rotation merging step but uses a heuristic approach to complete the parity network without simultaneously optimizing the phase rotation and output state transformations. While effective for small circuits, its benefits diminish for larger circuits compared to ours.

In contrast, our PhasePoly technique co-optimizes phase parity network synthesis and output state transformation circuit synthesis,

Circuit	Org.	GRAY-SYNTH	QUESO	PhasePoly.	QUESO+PhasePoly.
	# Gates CXs	# CXs	# Gates CXs	# Gates CXs	# Gates CXs
rc_adder_6	200 93	71	176 79	152 71	152 71
tof_10	255 102	70	175 70	175 70	175 70
hwb6	259 116	110	218 103	200 96	199 95
mod_red_21	278 105	86	198 79	180 77	179 76
qaoa_n8_p4	440 96	-	244 92	240 88	240 88
ham15-low	443 236	208	343 200	336 198	333 195
qcla_com_7	443 186	136	295 127	256 120	256 120
barenco_tof_10	450 192	144	262 128	248 114	248 114
qcla_adder_10	521 233	214	443 207	391 176	391 176
grover_5	831 288	226	589 208	455 202	455 202
qcla_mod_7	884 382	360	778 356	628 289	620 281
adder_8	900 409	359	628 301	565 270	561 270
ham15-med	1272 534	357	773 374	658 327	654 323
mod_adder_1024	4285 1720	1390	2837 1237	2565 1217	2550 1202
ham15-high	5308 2149	1502	4653 2134	2792 1363	2786 1357
shor_15_7	36598 14858	-	34969 13308	31143 12814	31143 12814
Geo. Mean Reduction	-	16.70%	26.34% 19.96%	35.61% 27.46%	35.83% 27.90%

Table 2: Reduction improvement of combined PhasePoly with another optimization pass. Our PhasePoly optimization not only outperforms the 2-qubit gate framework Gray-Synth as an independent optimization approach, but it also delivers the best performance when integrated as an orthogonal approach with the equivalent-circuit rewriting framework QUESO.

achieving up to a 50% reduction in total gate count and a 48.57% reduction in CNOT count. On average, it reduces the total gate count and CNOT count by 34.92% and 28.54%, outperforming Rotation Merging and MST-based parity network synthesis by 16.62% and 11.62%, respectively.

4.2.2 Evaluation of Integration with Other Optimization Frameworks. Our PhasePoly optimization framework not only excels in phase polynomial optimization, but also is an orthogonal optimization approach that can be integrated with existing equivalent-circuit-rewriting passes. Table 2 compares its performance regarding total gate count and CNOT count. We focus on circuits with at least 200 gates to showcase its long-range optimization capabilities, as smaller circuits—easily optimized by local search—lack the necessary complexity. Moreover, among the nine circuits with fewer than 200 gates that were excluded, PhasePoly either outperformed or matched other methods, demonstrating its benefits across a broad range of circuit sizes.

We first report that the CNOT gate count data is derived from the prior 2-qubit gate optimization framework, GRAY-SYNTH [2], which incorporates additional optimization approaches for T gate reduction and CCZ decomposition techniques [3]. Despite using a fixed-rule decomposed circuit with a higher number of R_z gates, our approach outperforms or matches GRAY-SYNTH across all available benchmark circuits not only in CNOT gate count but also in total gate count.

Next, we integrate PhasePoly with QUESO and compare the results of QUESO alone with those of the combined QUESO+PhasePoly workflow. While QUESO achieves results nearly equivalent to PhasePoly on small-range circuits, the gate count of integration with PhasePoly significantly reduces when facing larger circuits. This improved optimization enables more effective use of equivalent-circuit rewriting techniques, leading to substantial gains: the total gate reduction rate increases from 26.34% to 35.83%, and the CNOT gate reduction rate rises from 19.96% to 27.90%.

With PhasePoly as an orthogonal step, all benchmark circuits achieve equal or improved gate count reductions compared to any independent optimization approach. Larger circuits benefit the most, where local equivalent-circuit rewriting techniques often struggle. For instance, the ham15-high circuit sees its CNOT count drop from 2134 to 1357 when moving from QUESO to QUESO+PhasePoly, boosting the CNOT reduction from 15 to 792, a 52.8x improvement. PhasePoly simplifies the circuit structure, allowing more effective rewrites by addressing inefficiencies in long-range rotation merging and phase parity network synthesis. Similarly, the mod_adder_1024 circuit experiences a 2375-gate reduction, lowering the total gate count by 40.49% after integrating PhasePoly with QUESO. These results confirm that PhasePoly significantly enhances optimization when integrated with other optimization passes.

4.3 Interaction with Hardware Circuits

After logical circuits are optimized, we can run physical circuit compilation on that. A good logical circuit with fewer 2-qubit gates is correlated with a good physical circuit after hardware qubit mapping. Hence, one scenario we evaluate is that of running a qubit mapper Sabre [31] on top of the optimized circuits. This is shown in Fig. 11. In the meantime, we also show a phase-polynomial-only synthesis approach that is totally SWAP free by Steiner tree [55]. In Fig. 11, the gate count is normalized by setting the result of the **Original + Sabre** method, which is hardware mapping without any logical optimization. The approach using the Steiner Tree synthesis method is generally inefficient because it does not allow qubit layout changes. The third (yellow) bar represents that QUESO logical circuit optimization is used prior to the Sabre mapper. By reducing gates at the logical level, this method yields better results than the **Original + Sabre** in 76.47% of the benchmarks. In the **QUESO + PhasePoly + Sabre** approach, we see even more reduction in CNOT gates, leading to improvements over the baseline in all but the qaoa_n8_p4 circuit. However, in this case, we noticed the Queso

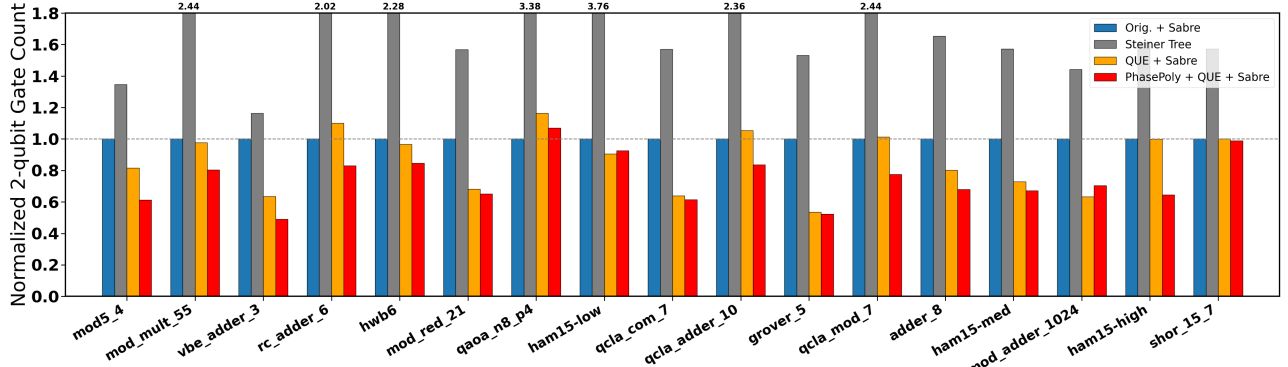


Figure 11: Normalized 2-qubit gate count comparison of different physical circuit compilation approaches.

+ Sabre approach does not improve over the baseline either, and PhasePoly + Queso + Sabre is better. Overall this integrated method is the best approach in 82.35% of cases, achieving an average 2-qubit gate count reduction of 25.47% compared to the baseline.

PhasePoly also enables targeted optimization for circuits with recurring building blocks. As introduced in Section 2.3, for limited connectivity hardware, PhasePoly can generate a hardware-friendly subcircuit template with linear connectivity requirements. Beyond the example in Section 2.3, we identified two additional benchmark circuits that benefit from hardware-friendly subcircuit template generation: the Adder circuit implemented by Vedral *et al.* [57] and the V-chain Multi-control NOT circuit from Nielsen and Chuang [44]. We extracted recurring 3-qubit or 4-qubit components from these circuits and generated their linear templates, which were then embedded into the original circuits. The final circuit was compiled using Sabre, and its performance was compared with the **Original + Sabre** and Queso + Sabre approaches, as shown in Figure 12.

For the Adder circuit, PhasePoly + Sabre significantly outperforms Queso + Sabre approach, when the qubit number is large, showcasing its scalability. For the V chain Multi-control NOT circuit, PhasePoly + Sabre results are comparable to those of QUESO+Sabre, while consistently outperforming the **Original + Sabre** baseline. These experiments clearly demonstrate PhasePoly’s potential for hardware-friendly subcircuit template generation.

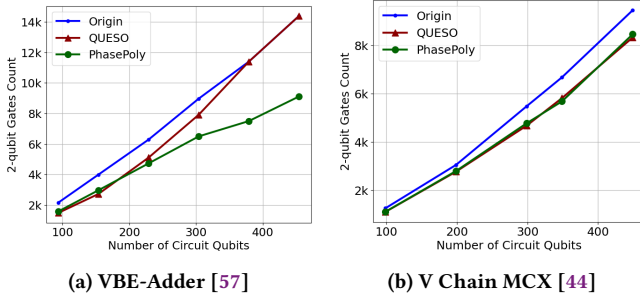


Figure 12: Comparison of VBE-Adder and V Chain MCX circuits optimized using Only Mapping, Queso + Mapping, and Poly Hardware-Friendly Circuit Generation approaches.

4.4 Sensitivity Analysis in Multi-Block Phase Polynomial Optimization

For the sensitivity analysis of multi-block phase polynomial optimization, we evaluate the impact of different group sizes on the optimization performance. Three different circuits are selected for this analysis, as shown in Table 3.

For the barenco_10 circuit, Groups 1 and 2 yield similar results, with notable improvements only at Group 3, while for adder_8 the performance continues to improve as the group size increases—indicating that limiting the group to a single block might miss a 5-10% gain. In contrast, the ham15_low circuit shows declining performance with larger groups, likely due to the expanded search space. To address these, we employ a progressive synthesis approach that first optimizes a single block and then uses that result to constrain subsequent multi-block optimizations, with future work aimed at developing faster methods for determining the optimal group size with less time needed.

Circuit	adder_8		barenco_10		ham15_low	
	# GATES	# CXs	# GATES	# CXs	# GATES	# CXs
Org.	900	409	450	192	443	236
Group 1	580	285	262	128	340	202
Group 2	572	277	262	128	344	206
Group 3	567	272	248	114	351	213
Group 5	565	270	248	114	356	218

Table 3: Reduction improvement with multi-block phase polynomial optimization. For adder_8, the best performance is achieved with Group 5; for barenco_10, Group 3 performs as well as Group 5; and for ham15_low, optimal performance is reached with single block setting, declining when larger group size settings are used.

4.5 Compilation Efficiency Analysis

To analyze compilation efficiency between phase polynomial optimization and the equivalent-circuit rewriting optimization passes, we compared our method with the QUESO framework under varying time budgets across three benchmarks (150-900 gates), as shown in Table 4. For smaller benchmarks, our technique reaches near-optimal optimization in 15-60 seconds—completing in 5 seconds for some cases on an Intel Xeon CPU—while QUESO’s improvements remain limited even at a 3600-second budget. With our default phase

polynomial optimization using a 400-second budget, our approach outperforms QUESO (using 3600 seconds) in 76.83% of benchmarks and performs at least as well in 79.16%. Furthermore, complete multi-block optimizations match or exceed QUESO’s results in 96% of benchmarks. Overall, PhasePoly consistently achieves superior gate count reductions in a fraction of QUESO’s runtime, making it highly time-efficient, especially under tight time constraints.

Circuit	Techniques	0s	15s	60s	600s	3600s
vbe_adder_3	PhasePoly	150	75	75	75	75
	QUESO	150	101	87	86	85
barenco_tof_10	PhasePoly	450	262	248	248	248
	QUESO	450	416	294	262	262
adder_8	PhasePoly	900	587	580	565	565
	QUESO	900	865	737	656	628

Table 4: Comparison of total gate counts produced by PhasePoly and QUESO under different optimization time budgets. Each column shows the resulting gate count after applying each optimization technique with a time budget.

5 Related Work

Rule-set Rewrite Circuit Optimization. Most previous studies focus on the local rewriting of gate sequences for circuit optimization using a rule-set approach. These studies [20, 42, 51, 53] search for subcircuit rewrite opportunities and perform a series of rewrites until no further optimization is possible. Some use manually constructed rule-set databases, while others, like Quanto [46], Queso [59], and Quartz [60], automatically generate rule-sets. However, these rule-sets are limited to small subcircuit patterns with few gates. Other studies employ reinforcement learning [16, 34], but the extensive pre-training and limited gate transformations indicate early-stage development.

Phase Polynomial Synthesis. There are two key differences between our work and prior phase polynomial synthesis approaches. First, previous studies [3, 55] focus on phase parity optimization while leaving output parity to traditional engines. Nam *et al.* [42] address both, but their work only merges and floats rotations, missing the full potential of phase polynomials. Other studies [3, 58] focus on T gate for fault-tolerant computation but not CNOT.

Qubit Mapping Optimization. General qubit mapping strategies [1, 6, 12, 18, 22, 30, 31, 41, 50, 61, 62, 64, 65, 68] address limited qubit connectivity but don’t optimize logical circuits. Other compilers [23, 29] optimize logical circuits for mitigating qubit connectivity constraints, but they do not reduce the two-qubit gate count. Vandaele *et al.* [55] co-designs phase polynomial synthesis and qubit mapping using the Steiner tree but lacks flexibility in qubit layout. Some VQE frameworks [24, 32, 33] integrate both logical synthesis and qubit mapping, but only for Pauli-string optimizations.

6 Conclusion

Quantum circuit optimization is critical for selecting the best circuit implementation for a given algorithm. This paper explores the role of phase-polynomial optimization in full-circuit optimization and improves upon the state-of-the-art without requiring an exhaustive search. Additionally, we provide an approach to deliver hardware-friendly recurring phase polynomial blocks.

References

- [1] Gadi Aleksandrowicz, Thomas Alexander, Panagiotis Barkoutsos, Luciano Bello, Yael Ben-Haim, David Bucher, Francisco Jose Cabrera-Hernández, Jorge Carballo-Franquis, Adrian Chen, Chun-Fu Chen, Jerry M. Chow, Antonio D. Córcoles-Gonzales, Abigail J. Cross, Andrew Cross, Juan Cruz-Benito, Chris Culver, Salvador De La Puente González, Enrique De La Torre, Delton Ding, Eugene Dumitrescu, Ivan Duran, Pieter Eendebak, Mark Everitt, Ismael Faro Sertage, Albert Frisch, Andreas Fuhrer, Jay Gambetta, Borja Godoy Gago, Juan Gomez-Mosquera, Donny Greenberg, Ikko Hamamura, Vojtech Havlicek, Joe Hellmers, Łukasz Herok, Hiroshi Horii, Shaohan Hu, Takashi Imamichi, Toshinari Itoko, Ali Javadi-Abhari, Naoki Kanazawa, Anton Karazeev, Kevin Krsulich, Peng Liu, Yang Luh, Yunho Maeng, Manoel Marques, Francisco Jose Martin-Fernández, Douglas T. McClure, David McKay, Srujan Meesala, Antonio Mezzacapo, Nikolaj Moll, Diego Moreda Rodríguez, Giacomo Nannicini, Paul Nation, Pauline Olitrault, Lee James O’Riordan, Hanhee Paik, Jesús Pérez, Anna Phan, Marco Pistoia, Viktor Prutyayov, Max Reuter, Julia Rice, Abdón Rodríguez Davila, Raymond Harry Putra Rudy, Mingi Ryu, Ninad Sathaye, Chris Schnabel, Eddie Schoute, Kanav Setia, Yunong Shi, Adenilton Silva, Yukio Siraichi, Seyon Sivarajah, John A. Smolin, Mathias Soeken, Hitomi Takahashi, Ivano Tavernelli, Charles Taylor, Pete Taylour, Kenso Trabing, Matthew Treinish, Wes Turner, Desiree Vogt-Lee, Christophe Vuillot, Jonathan A. Wildstrom, Jessica Wilson, Erick Winston, Christopher Wood, Stephen Wood, Stefan Wörner, Ismail Yunus Akhalwaya, and Christa Zoufal. 2019. *Qiskit: An Open-source Framework for Quantum Computing*. <https://doi.org/10.5281/zenodo.2562111>
- [2] Matthew Amy, Parsiad Azimzadeh, and Michele Mosca. 2018. On the controlled-NOT complexity of controlled-NOT-phase circuits. *Quantum Science and Technology* 4, 1 (2018), 015002.
- [3] Matthew Amy, Dmitri Maslov, and Michele Mosca. 2014. Polynomial-time T-depth optimization of Clifford+ T circuits via matroid partitioning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 33, 10 (2014), 1476–1489.
- [4] Juan Miguel Arrazola, Olivia Di Matteo, Nicolás Quesada, Soran Jahangiri, Alain Delgado, and Nathan Killoran. 2022. Universal quantum circuits for quantum chemistry. *Quantum* 6 (2022), 742.
- [5] Dominic W. Berry, Graeme Ahokas, Richard Cleve, and Barry C. Sanders. 2006. Efficient Quantum Algorithms for Simulating Sparse Hamiltonians. *Communications in Mathematical Physics* 270, 2 (Dec. 2006), 359–371. <https://doi.org/10.1007/s00220-006-0150-x>
- [6] Kyle Booth, Minh Do, J Beck, Eleanor Rieffel, Davide Venturelli, and Jeremy Frank. 2018. Comparing and integrating constraint programming and temporal planning for quantum circuit compilation. In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 28. 366–374.
- [7] Lukas Burgholzer and Robert Wille. 2020. Advanced equivalence checking for quantum circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 9 (2020), 1810–1824.
- [8] Yudong Cao, Jonathan Romero, Jonathan P Olson, Matthias Degroote, Peter D Johnson, Mária Kieferová, Ian D Kivlichan, Tim Menke, Borja Peropadre, Nicolas PD Sawaya, et al. 2019. Quantum chemistry in the age of quantum computing. *Chemical reviews* 119, 19 (2019), 10856–10915.
- [9] Christopher M Dawson, Henry L Haselgrove, Andrew P Hines, Duncan Mortimer, Michael A Nielsen, and Tobias J Osborne. 2004. Quantum computing and polynomial equations over the finite field $\mathbb{Z}_2$. *arXiv preprint quant-ph/0408129* (2004).
- [10] Timothée Goubault De Brugière, Marc Baboulin, Benoît Valiron, Simon Martiel, and Cyril Allouche. 2021. Gaussian elimination versus greedy methods for the synthesis of linear reversible circuits. *ACM Transactions on Quantum Computing* 2, 3 (2021), 1–26.
- [11] Jonathan HA de Carvalho and Fernando M de Paula Neto. 2023. Parametrized constant-depth quantum neuron. *IEEE Transactions on Neural Networks and Learning Systems* (2023).
- [12] Haowei Deng, Yu Zhang, and Quanxi Li. 2020. Codar: A contextual duration-aware qubit mapping for various nqsq devices. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [13] Casey Duckering, Jonathan M Baker, Andrew Litteken, and Frederic T Chong. 2021. Orchestrated trios: compiling for efficient communication in quantum programs with 3-qubit gates. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 375–385.
- [14] Edward Farhi, Jeffrey Goldstone, and Sam Gutmann. 2014. A Quantum Approximate Optimization Algorithm. *arXiv:1411.4028 [quant-ph]*
- [15] Richard P Feynman. 2018. Simulating physics with computers. In *Feynman and computation*. cRc Press, 133–153.
- [16] Thomas Fösel, Murphy Yuezheng Niu, Florian Marquardt, and Li Li. 2021. Quantum circuit optimization with deep reinforcement learning. *arXiv preprint arXiv:2103.07585* (2021).
- [17] Jay Gambetta. 2023. The hardware and software for the era of quantum utility is here.

- [18] Xiangyu Gao, Yuwei Jin, Minghao Guo, Henry Chen, and Eddy Z. Zhang. 2024. Linear Depth QFT over IBM Heavy-hex Architecture. *arXiv:2402.09705* [quant-ph] <https://arxiv.org/abs/2402.09705>
- [19] Lov K. Grover. 1996. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing* (Philadelphia, Pennsylvania, USA) (STOC '96). Association for Computing Machinery, New York, NY, USA, 212–219. <https://doi.org/10.1145/237814.237866>
- [20] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. 2021. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages* 5, POPL (2021), 1–29.
- [21] Raban Iten, Roger Colbeck, Ivan Kukuljan, Jonathan Home, and Matthias Christandl. 2016. Quantum circuits for isometries. *Physical Review A* 93, 3 (2016), 032318.
- [22] Yuwei Jin, Xiangyu Gao, Minghao Guo, Henry Chen, Fei Hua, Chi Zhang, and Eddy Z. Zhang. 2024. Optimizing Quantum Fourier Transformation (QFT) Kernels for Modern NISQ and FT Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis* (Atlanta, GA, USA) (SC '24). IEEE Press, Article 68, 15 pages. <https://doi.org/10.1109/SC41406.2024.00074>
- [23] Yuwei Jin, Fei Hua, Yanhao Chen, Ari Hayes, Chi Zhang, and Eddy Z. Zhang. 2024. Exploiting the Regular Structure of Modern Quantum Architectures for Compiling and Optimizing Programs with Permutable Operators. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4* (Vancouver, BC, Canada) (ASPLOS '23). Association for Computing Machinery, New York, NY, USA, 108–124. <https://doi.org/10.1145/3623278.3624751>
- [24] Y. Jin, Z. Li, F. Hua, T. Hao, H. Zhou, Y. Huang, and E. Z. Zhang. 2024. Tetris: A Compilation Framework for VQA Applications in Quantum Computing. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, Los Alamitos, CA, USA, 277–292. <https://doi.org/10.1109/ISCA59077.2024.00029>
- [25] Hermann Kaindl and Aliasghar Khorsand. 1994. Memory-bounded bidirectional search. In *AAAI*. 1359–1364.
- [26] Aleks Kissinger and Arianne Meijer van de Griend. 2019. CNOT circuit extraction for topologically-constrained quantum memories. *arXiv:1904.00633* [quant-ph] <https://arxiv.org/abs/1904.00633>
- [27] Aleks Kissinger and John van de Wetering. 2019. PyZX: Large scale automated diagrammatic reasoning. *arXiv preprint arXiv:1904.04735* (2019).
- [28] Dax Enshan Koh, Mark D Penney, and Robert W Spekkens. 2017. Computing quipit Clifford circuit amplitudes by the sum-over-paths technique. *arXiv preprint arXiv:1702.03316* (2017).
- [29] Lingling Lao and Dan E Browne. 2022. 2QAN: A Quantum Compiler for 2-Local Qubit Hamiltonian Simulation Algorithms. (2022), 351–365. <https://doi.org/10.1145/3470496.3527394>
- [30] Lingling Lao, Hans Van Someren, Imran Ashraf, and Carmen G Almudever. 2021. Timing and resource-aware mapping of quantum circuits to superconducting processors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 2 (2021), 359–371.
- [31] Gushu Li, Yufei Ding, and Yuan Xie. 2019. Tackling the qubit mapping problem for NISQ-era quantum devices. *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (2019), 1001–1014.
- [32] Gushu Li, Yunong Shi, and Ali Javadi-Abhari. 2021. Software-hardware co-optimization for computational chemistry on superconducting quantum processors. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 832–845.
- [33] Gushu Li, Anbang Wu, Yunong Shi, Ali Javadi-Abhari, Yufei Ding, and Yuan Xie. 2022. Paulihedral: A Generalized Block-Wise Compiler Optimization Framework for Quantum Simulation Kernels. *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (2022), 554–569. <https://doi.org/10.1145/3503222.3507715>
- [34] Zikun Li, Jinjun Peng, Yixuan Mei, Sina Lin, Yi Wu, Oded Padon, and Zhihao Jia. 2024. Quarl: A learning-based quantum circuit optimizer. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 555–582.
- [35] Seth Lloyd. 1996. Universal Quantum Simulators. *Science* 273 (1996), 1073–1078. Issue 5278. <https://doi.org/10.1126/science.273.5278.1073>
- [36] Emanuel Malvetti, Raban Iten, and Roger Colbeck. 2021. Quantum circuits for sparse isometries. *Quantum* 5 (2021), 412.
- [37] Sam McArdle, Suguru Endo, Alan Aspuru-Guzik, Simon C Benjamin, and Xiao Yuan. 2020. Quantum computational chemistry. *Reviews of Modern Physics* 92, 1 (2020), 015003.
- [38] Arianne Meijer-van de Griend and Ross Duncan. 2023. Architecture-Aware Synthesis of Phase Polynomials for NISQ Devices. *Electronic Proceedings in Theoretical Computer Science* 394 (Nov. 2023), 116–140. <https://doi.org/10.4204/eptcs.394.8>
- [39] D. Michael Miller, Dmitri Maslov, and Gerhard W. Dueck. 2003. A transformation based algorithm for reversible logic synthesis. In *Proceedings of the 40th Annual Design Automation Conference* (Anaheim, CA, USA) (DAC '03). Association for Computing Machinery, New York, NY, USA, 318–323. <https://doi.org/10.1145/775832.775915>
- [40] Ashley Montanaro. 2016. Quantum circuits and low-degree polynomials over $\mathbb{F}_2$. *arXiv preprint arXiv:1607.08473* (2016).
- [41] Prakash Murali, Jonathan M. Baker, Ali Javadi Abhari, Frederic T. Chong, and Margaret Martonosi. 2019. Noise-Adaptive Compiler Mappings for Noisy Intermediate-Scale Quantum Computers. *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems* (4 2019), 1015–1029. <https://doi.org/10.1145/3297858.3304075>
- [42] Yunseong Nam, Neil J Ross, Yuan Su, Andrew M Childs, and Dmitri Maslov. 2018. Automated optimization of large quantum circuits with continuous parameters. *npj Quantum Information* 4, 1 (2018), 23.
- [43] Beatrice Nash, Vlad Gheorghiu, and Michele Mosca. 2020. Quantum circuit optimizations for NISQ architectures. *Quantum Science and Technology* 5, 2 (March 2020), 025010. <https://doi.org/10.1088/2058-9565/ab79b1>
- [44] Michael A Nielsen and Isaac Chuang. 2002. Quantum computation and quantum information.
- [45] Ketan N Patel, Igor L Markov, and John P Hayes. 2008. Optimal synthesis of linear reversible circuits. *Quantum Inf. Comput.* 8, 3 (2008), 282–294.
- [46] Jessica Pointing, Oded Padon, Zhihao Jia, Henry Ma, Auguste Hirth, Jens Palsberg, and Alex Aiken. 2021. Quanto: Optimizing quantum circuits with automatic generation of circuit identities. *Quantum Science and Technology* (2021).
- [47] Francisco JR Ruiz, Tuomas Laakkonen, Johannes Bausch, Matej Balog, Mohammadamin Barekatain, Francisco JH Heras, Alexander Novikov, Nathan Fitzpatrick, Bernardino Romera-Paredes, John van de Wetering, et al. 2025. Quantum circuit optimization with alphasensor. *Nature Machine Intelligence* (2025), 1–12.
- [48] Peter W. Shor. 1994. Algorithms for quantum computation: Discrete logarithms and factoring. *Proceedings - Annual IEEE Symposium on Foundations of Computer Science, FOCS* (1994). <https://doi.org/10.1109/SFCS.1994.365700>
- [49] Peter W Shor. 1999. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review* 41, 2 (1999), 303–332.
- [50] Animesh Sinha, Utkarsh Azad, and Harjinder Singh. 2022. Qubit routing using graph neural network aided Monte Carlo tree search. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 9935–9943.
- [51] Seyon Sivarajah, Silas Dilkes, Alexander Cowtan, Will Simmons, Alec Edgington, and Ross Duncan. 2020. $\mathbf{t|ket|}$: a retargetable compiler for NISQ devices. *Quantum Science and Technology* 6, 1 (2020), 014003.
- [52] Mark Skilbeck, Eric Peterson, appleby, Erik Davis, Peter Karalekas, Juan M. Bello-Rivas, Daniel Kochmanski, Zach Beane, Robert Smith, Andrew Shi, Cole Scott, Adam Paszke, Eric Hulburd, Matthew Young, Aaron S. Jackson, BHAVISHYA, M. Sohaib Alam, Wilfredo Velázquez-Rodríguez, c. b. osborn, fengdlm, and jmackeyrigetti. 2020. *rigetti/quilc: v1.21.0*. <https://doi.org/10.5281/zenodo.3967926>
- [53] Robert S Smith, Eric C Peterson, Mark G Skilbeck, and Erik J Davis. 2020. An open-source, industrial-strength optimizing compiler for quantum programs. *Quantum Science and Technology* 5, 4 (2020), 044001.
- [54] Andrei Tănăsescu, David Constantinescu, and Pantelimon George Popescu. 2022. Distribution of controlled unitary quantum gates towards factoring large numbers on today's small-register devices. *Scientific Reports* 12, 1 (2022), 21310.
- [55] Vivien Vandaele, Simon Martiel, and Timothée Goubault de Brugière. 2022. Phase polynomials synthesis algorithms for NISQ architectures and beyond. *Quantum Science and Technology* 7, 4 (2022), 045027.
- [56] Vlatko Vedral, Adriano Barenco, and Artur Ekert. 1996. Quantum networks for elementary arithmetic operations. *Physical Review A* 54, 1 (jul 1996), 147–153. <https://doi.org/10.1103/physreva.54.147>
- [57] Vlatko Vedral, Adriano Barenco, and Artur Ekert. 1996. Quantum networks for elementary arithmetic operations. *Physical Review A* 54, 1 (1996), 147.
- [58] Hassler Whitney. 1992. On the abstract properties of linear dependence. *Hassler Whitney Collected Papers* (1992), 147–171.
- [59] Amanda Xu, Abtin Molavi, Lauren Pick, Swamit Tannu, and Aws Albarghouthi. 2023. Synthesizing quantum-circuit optimizers. *Proceedings of the ACM on Programming Languages* 7, PLDI (2023), 835–859.
- [60] Mingkuan Xu, Zikun Li, Oded Padon, Sina Lin, Jessica Pointing, Auguste Hirth, Henry Ma, Jens Palsberg, Alex Aiken, Umut A Acar, et al. 2022. Quartz: superoptimization of quantum circuits. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 625–640.
- [61] Chi Zhang, Yanhao Chen, Yuwei Jin, Wonsun Ahn, Youtao Zhang, and Eddy Z Zhang. 2020. A Depth-Aware Swap Insertion Scheme for the Qubit Mapping Problem. *arXiv preprint arXiv:2002.07289* (2020).
- [62] Chi Zhang, Ari B Hayes, Longfei Qiu, Yuwei Jin, Yanhao Chen, and Eddy Z Zhang. 2021. Time-optimal qubit mapping. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 360–374.
- [63] Rong Zhou and Eric A Hansen. 2002. Memory-Bounded A* Graph Search.. In *FLAIRS*. 203–209.
- [64] Xiangzhen Zhou, Yuan Feng, and Sanjiang Li. 2020. A Monte Carlo tree search framework for quantum circuit transformation. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–7.

- [65] Xiangzhen Zhou, Sanjiang Li, and Yuan Feng. 2020. Quantum circuit transformation based on simulated annealing and heuristic search. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 4683–4694.
- [66] Ben Zindorf and Sougato Bose. 2024. Efficient implementation of multi-controlled quantum gates. *arXiv preprint arXiv:2404.02279* (2024).
- [67] Henry Zou, Matthew Treinish, Kevin Hartman, Alexander Ivrii, and Jake Lishman. 2024. LightSABRE: A Lightweight and Enhanced SABRE Algorithm. *arXiv preprint arXiv:2409.08368* (2024).
- [68] Alwin Zulehner, Alexandru Paler, and Robert Wille. 2018. An efficient methodology for mapping quantum circuits to the IBM QX architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 7 (2018), 1226–1236.

DISCLAIMER

This paper was prepared for informational purposes by the Global Technology Applied Research center of JPMorgan Chase & Co. This

paper is not a product of the Research Department of JPMorgan Chase & Co. or its affiliates. Neither JPMorgan Chase & Co. nor any of its affiliates makes any explicit or implied representation or warranty and none of them accept any liability in connection with this paper, including, without limitation, with respect to the completeness, accuracy, or reliability of the information contained herein and the potential legal, compliance, tax, or accounting effects thereof. This document is not intended as investment research or investment advice, or as a recommendation, offer, or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction.